Week 5 Submission

Peer-to-Peer Lending Matchmaking System

Regd No	25091A32G9, 25091A32K4
Problem Statement No	105
Title	Peer-to-Peer Lending Matchmaking System
Technology	Python, Flask, SQLite, NetworkX, Pandas/NumPy, ReportLab, OpenPyXL, Chart.js, Bootstrap, unittest/pytest
Data	Academic simulation using simulated test data - no real-money transactions

Where to run the code (read this first)

- **Where:** on your own laptop. Install **Python 3.10 or newer** from python.org (tick *Add Python to PATH*) and **VS Code**. Everything below is typed in the **VS Code terminal** (menu Terminal > New Terminal, or Ctrl+`).
- **Which folder:** open your **p2p_lending** folder (the one that contains **app.py**) with File > Open Folder. Every command must be run from this folder, not from a sub-folder, not in IDLE.
- **One-time setup:** run `python -m venv venv`, then activate it - Windows: `venv\Scripts\activate` (if PowerShell blocks it use Command Prompt or run `Set-ExecutionPolicy -Scope Process Bypass`), macOS/Linux: `source venv/bin/activate`. Then `pip install -r requirements.txt`.
- **Adding the files of a week:** in VS Code right-click the folder > New File, type the exact path shown in the heading (for example `routes/profiles.py`), paste the code. **[NEW]** = create the file, **[REPLACE]** = overwrite the old file completely, **[APPEND]** = paste at the very end of the existing file.
- **Web pages:** run `python app.py`, then open `http://127.0.0.1:5000` in your browser (stop the server with Ctrl+C). The pages load Bootstrap/Chart.js from the internet, so stay online.
- **Keep the weeks cumulative:** Week N needs the files of Weeks 1 to N-1. Your Week 1 zip is the starting point and none of its files is deleted later (only app.py and templates/index.html are replaced in Week 2, and config.py gets lines appended in Week 3).
- **Demo scripts** (`demo_weekN.py`) create their own throw-away databases in the data folder, so they never disturb your real database. To see the same demo data in the web pages: stop the server, delete `data/p2p_lending.db`, run `python demo_data.py`, start the server again.

Week 5: Loan and Repayment Simulation

Convert accepted matches into simulated loans, generate instalment schedules, simulate paid / late / defaulted repayments, update outstanding balances and record every status change in the audit log. No real payments are used. **Working output: loan lifecycle tracker.**

Assignment steps and where they are done

Assignment step	Implemented in
1. Convert accepted matches into simulated loans	loan_service.accept_request() / accept_all(); page /loans/pending
2. Store principal amount, rate and duration	loans table: principal, interest_rate (from the borrower's risk class), duration_months
3. Generate an instalment schedule	build_schedule(): reducing-balance EMI, last instalment clears the balance
4. Record repayment due dates	repayments table: monthly due dates (month-end safe)
5. Simulate repayment statuses	simulate_loan(): all_paid, some_late, default, random (probabilities by risk class); mark_installment() for manual changes
6. Update outstanding balances	recalculated after every instalment; loan becomes completed or defaulted
7. Record every status change in the audit log	audit.log() inside accept_request() and mark_installment()
8. No real payments	everything is simulated inside the database

Files of this week

File	Action	Purpose
loan_service.py	NEW	Schedules, accepting matches, repayment statuses, simulation
routes/loans.py	NEW	/loans/pending, /loans, /loans/<id> (schedule with Paid / Late / Default buttons)
demo_week5.py	NEW	Console demo
tests/test_week5.py	NEW	14 tests

Rules of the simulation

Event	Rule
Accepting a request	Only 'matched' requests; every lender must still hold the allocated amount; all-or-nothing (database rollback on error); lender funds are reduced; request becomes 'funded'
Paid	amount_paid = amount_due on the due date
Late	paid 7 days after the due date with a 2% late fee added to amount_paid
Defaulted	amount_paid = 0, all later instalments are written off as defaulted, loan status becomes 'defaulted'
Order	instalments must be settled in order; a settled instalment cannot be changed again
Outstanding balance	principal left according to the schedule after the settled instalments (0 when completed)

How to run and check this week

1. `python tests/test_week5.py` - 14 tests, all ok
2. `python demo_week5.py` - loans, schedule, paid / late / default cases, lender funds, audit log
3. `python app.py` - run matching, open /loans/pending, press 'Accept all', then open /loans and a loan's schedule

Expected output of demo_week5.py

```
Matching: {'lenders': 5, 'requests': 7, 'eligible_edges': 11, 'recommended': 5, 'requests_matched': 5, 'allocation_success_rate': 71.4,
    'total_allocated': 285000.0, 'partial_funding': False, 'seconds': 0.0086}

Accepted matches -> 5 loans created. Errors: []

=== Loans ===
Loan #1 Conservative Carla  -> Ravi Kumar   principal=   50,000 rate=9% months=12 outstanding=   50,000
Loan #2 Aggressive Arjun    -> Meena Iyer   principal=   20,000 rate=20% months= 6 outstanding=   20,000
Loan #3 Balanced Bala       -> Sunil Verma  principal=   80,000 rate=14% months=24 outstanding=   80,000
Loan #4 Conservative Carla  -> Kiran Rao    principal=   15,000 rate=9% months=12 outstanding=   15,000
Loan #5 Conservative Carla  -> Kiran Rao    principal=  120,000 rate=9% months=36 outstanding=  120,000

=== First 4 instalments of loan #1 ===
  #1 due 2026-02-15 amount  4,372.57 [pending]
  #2 due 2026-03-15 amount  4,372.57 [pending]
  #3 due 2026-04-15 amount  4,372.57 [pending]
  #4 due 2026-05-15 amount  4,372.57 [pending]

=== Loan 1: all instalments paid ===
{'loan_id': 1, 'match_id': 1, 'lender_id': 1, 'lender': 'Conservative Carla', 'borrower_id': 1, 'borrower': 'Ravi Kumar', 'principal':
    50000.0, 'interest_rate': 9.0, 'duration_months': 12, 'outstanding_balance': 0.0, 'status': 'completed', 'start_date': '2026-01-15',
    'settled': 12}

=== Loan 2: some instalments late ===
{'loan_id': 2, 'status': 'completed', 'outstanding_balance': 0.0, 'settled': 6}
[(1, 'paid', 3530.46), (2, 'paid', 3530.46), (3, 'paid', 3530.46), (4, 'late', 3601.07)]

=== Loan 3: borrower defaults halfway ===
{'loan_id': 3, 'status': 'defaulted', 'outstanding_balance': 42779.3, 'settled': 12}
['defaulted', 'paid']
=== Lender funds after loans were created ===
  Conservative Carla  available now    115,000
  Balanced Bala       available now    170,000
  Aggressive Arjun    available now    230,000
  Small Sita          available now     20,000
  Cohort Chitra       available now    100,000

=== Audit log (last 6 entries) ===
  ('loan', 3, 'STATUS_CHANGE', 'active', 'defaulted | outstanding balance 42,779.30')
  ('repayment', 42, 'STATUS_CHANGE', 'pending', 'defaulted | loan defaulted - remaining instalments written off')
  ('repayment', 41, 'STATUS_CHANGE', 'pending', 'defaulted | loan defaulted - remaining instalments written off')
  ('repayment', 40, 'STATUS_CHANGE', 'pending', 'defaulted | loan defaulted - remaining instalments written off')
  ('repayment', 39, 'STATUS_CHANGE', 'pending', 'defaulted | loan defaulted - remaining instalments written off')
  ('repayment', 38, 'STATUS_CHANGE', 'pending', 'defaulted | loan defaulted - remaining instalments written off')
Total audit entries: 90
```

[NEW] loan_service.py

```
"""Week 5 - convert accepted matches into simulated loans, build instalment
schedules, simulate repayment statuses and keep outstanding balances + audit log
up to date.  No real payments are involved.
"""
import calendar
import random
from datetime import date, datetime, timedelta

import audit
import config
import profile_service
from models import ValidationError, calculate_emi

SETTLED = ("paid", "late")


# ---------------------------------------------------------------------------
# schedule maths
# ---------------------------------------------------------------------------
def add_months(start, months):
    month_index = start.month - 1 + months
    year, month = start.year + month_index // 12, month_index % 12 + 1
    return date(year, month, min(start.day, calendar.monthrange(year, month)[1]))


def build_schedule(principal, annual_rate, months, start_date):
    """Reducing-balance schedule: list of dicts, last instalment clears the balance exactly."""
    emi = calculate_emi(principal, annual_rate, months)
    monthly_rate = annual_rate / 12 / 100
    balance, schedule = round(principal, 2), []
    for no in range(1, months + 1):
        interest = round(balance * monthly_rate, 2)
        principal_part = balance if no == months else round(emi - interest, 2)
        due = round(principal_part + interest, 2)
        balance = round(balance - principal_part, 2)
        schedule.append({"installment_no": no, "due_date": add_months(start_date, no).isoformat(),
                         "amount_due": due, "principal": principal_part, "interest": interest,
                         "balance_after": max(balance, 0.0)})
    return schedule


def _to_date(value):
    return value if isinstance(value, date) else datetime.strptime(str(value)[:10], "%Y-%m-%d").date()


def _rate_for_request(conn, request_id):
    row = conn.execute("SELECT risk_category FROM risk_scores WHERE request_id = ?", (request_id,)).fetchone()
    return config.RATE_BY_RISK[row["risk_category"] if row else "medium"]
def _category_for_request(conn, request_id):
```

```
    row = conn.execute("SELECT risk_category FROM risk_scores WHERE request_id = ?", (request_id,)).fetchone()
    return row["risk_category"] if row else "medium"


# ---------------------------------------------------------------------------
# accepting matches -> loans
# ---------------------------------------------------------------------------
def accept_request(conn, request_id, start_date=None, actor="admin"):
    """Turn every recommended match of a fully matched request into a loan.  Returns loan ids."""
    request = profile_service.get_request(conn, request_id)
    if request is None:
        raise ValidationError(f"Loan request #{request_id} does not exist.")
    if request.status != "matched":
        raise ValidationError(f"Request #{request_id} is '{request.status}', only 'matched' requests "
                              f"can be accepted.")
    matches = conn.execute("SELECT * FROM matches WHERE request_id = ? AND status = 'recommended'"
                           " AND allocated_amount > 0 ORDER BY match_id", (request_id,)).fetchall()
    if not matches:
        raise ValidationError(f"Request #{request_id} has no recommended matches.")
    if abs(sum(m["allocated_amount"] for m in matches) - request.amount) > 0.01:
        raise ValidationError(f"Request #{request_id} is not fully funded by its recommended matches.")
    start = _to_date(start_date) if start_date else date.today()
    rate = _rate_for_request(conn, request_id)
    borrower = profile_service.get_borrower(conn, request.borrower_id)
    try:
        for m in matches:                                       # never allocate beyond available funds
            lender = profile_service.get_lender(conn, m["lender_id"])
            if lender is None or not lender.can_fund(m["allocated_amount"]):
                raise ValidationError(f"Lender #{m['lender_id']} no longer has {m['allocated_amount']:,.2f} available.")
        loan_ids = []
        for m in matches:
            principal = m["allocated_amount"]
            cur = conn.execute(
                "INSERT INTO loans(match_id, lender_id, borrower_id, principal, interest_rate, duration_months,"
                " outstanding_balance, status, start_date) VALUES (?, ?, ?, ?, ?, ?, ?, 'active', ?)",
                (m["match_id"], m["lender_id"], request.borrower_id, principal, rate, request.term_months,
                 principal, start.isoformat()))
            loan_id = cur.lastrowid
            loan_ids.append(loan_id)
            conn.execute("UPDATE lenders SET available_amount = available_amount - ? WHERE lender_id = ?",
                         (principal, m["lender_id"]))
            for item in build_schedule(principal, rate, request.term_months, start):
                conn.execute("INSERT INTO repayments(loan_id, installment_no, due_date, amount_due) VALUES (?, ?, ?, ?)",
                             (loan_id, item["installment_no"], item["due_date"], item["amount_due"]))
            conn.execute("UPDATE matches SET status = 'accepted' WHERE match_id = ?", (m["match_id"],))
            audit.log(conn, "match", m["match_id"], "STATUS_CHANGE", "recommended", "accepted", actor=actor)
            audit.log(conn, "loan", loan_id, "LOAN_CREATED", None, "active", actor,
                      f"principal {principal:,.2f} at {rate:g}% for {request.term_months} months")
        conn.execute("UPDATE loan_requests SET status = 'funded' WHERE request_id = ?", (request_id,))
        audit.log(conn, "loan_request", request_id, "STATUS_CHANGE", "matched", "funded", actor=actor)
        conn.commit()
        return loan_ids
    except Exception:
        conn.rollback()
        raise


def accept_all(conn, start_date=None, actor="admin"):
    """Accept every matched request.  Returns (loan_ids, errors)."""
    loan_ids, errors = [], []
    for row in conn.execute("SELECT request_id FROM loan_requests WHERE status = 'matched' ORDER BY request_id").fetchall():
        try:
            loan_ids += accept_request(conn, row["request_id"], start_date, actor)
        except ValidationError as exc:
            errors.append(f"Request #{row['request_id']}: {exc}")
    return loan_ids, errors


# ---------------------------------------------------------------------------
# repayment status changes
# ---------------------------------------------------------------------------
def _outstanding(conn, loan):
    settled = conn.execute("SELECT COUNT(*) FROM repayments WHERE loan_id = ? AND status IN ('paid', 'late')",
                           (loan["loan_id"],)).fetchone()[0]
    if settled == 0:
        return round(loan["principal"], 2)
    schedule = build_schedule(loan["principal"], loan["interest_rate"], loan["duration_months"],
                              _to_date(loan["start_date"]))
    return schedule[settled - 1]["balance_after"]


def mark_installment(conn, repayment_id, status, actor="simulation"):
    """Set one instalment to paid / late / defaulted and update loan balance + audit log."""
    if status not in ("paid", "late", "defaulted"):
        raise ValidationError("Status must be 'paid', 'late' or 'defaulted'.")
    rep = conn.execute("SELECT * FROM repayments WHERE repayment_id = ?", (repayment_id,)).fetchone()
    if rep is None:
        raise ValidationError(f"Repayment #{repayment_id} does not exist.")
    if rep["status"] != "pending":
        raise ValidationError(f"Instalment is already '{rep['status']}'.")
    earlier = conn.execute("SELECT COUNT(*) FROM repayments WHERE loan_id = ? AND installment_no < ?"
                           " AND status = 'pending'", (rep["loan_id"], rep["installment_no"])).fetchone()[0]
    if earlier:
        raise ValidationError("Earlier instalments must be settled first.")
    loan = conn.execute("SELECT * FROM loans WHERE loan_id = ?", (rep["loan_id"],)).fetchone()
    due = _to_date(rep["due_date"])
    try:
        if status == "paid":
            conn.execute("UPDATE repayments SET status='paid', amount_paid=?, paid_date=? WHERE repayment_id=?",
                         (rep["amount_due"], due.isoformat(), repayment_id))
```

```
        elif status == "late":
            fee = round(rep["amount_due"] * config.LATE_FEE_RATE, 2)
            conn.execute("UPDATE repayments SET status='late', amount_paid=?, paid_date=? WHERE repayment_id=?",
                         (round(rep["amount_due"] + fee, 2), (due + timedelta(days=config.LATE_DAYS)).isoformat(),
                          repayment_id))
        else:
            conn.execute("UPDATE repayments SET status='defaulted', amount_paid=0 WHERE repayment_id=?",
                         (repayment_id,))
        audit.log(conn, "repayment", repayment_id, "STATUS_CHANGE", "pending", status, actor,
                  f"loan {loan['loan_id']} instalment {rep['installment_no']}")

        if status == "defaulted":                       # a default stops the whole loan
            for later in conn.execute("SELECT repayment_id FROM repayments WHERE loan_id = ? AND status = 'pending'",
                                      (loan["loan_id"],)).fetchall():
                conn.execute("UPDATE repayments SET status='defaulted' WHERE repayment_id = ?", (later[0],))
                audit.log(conn, "repayment", later[0], "STATUS_CHANGE", "pending", "defaulted", actor,
                          "loan defaulted - remaining instalments written off")
            new_status = "defaulted"
        else:
            pending = conn.execute("SELECT COUNT(*) FROM repayments WHERE loan_id = ? AND status = 'pending'",
                                   (loan["loan_id"],)).fetchone()[0]
            new_status = "completed" if pending == 0 else "active"
        balance = 0.0 if new_status == "completed" else _outstanding(conn, loan)
        conn.execute("UPDATE loans SET outstanding_balance = ?, status = ? WHERE loan_id = ?",
                     (balance, new_status, loan["loan_id"]))
        if new_status != loan["status"]:
            audit.log(conn, "loan", loan["loan_id"], "STATUS_CHANGE", loan["status"], new_status, actor,
                      f"outstanding balance {balance:,.2f}")
        conn.commit()
    except Exception:
        conn.rollback()
        raise


def simulate_loan(conn, loan_id, scenario="random", seed=None):
    """Simulate repayments of one loan.

    scenario: 'all_paid' | 'some_late' (every 4th instalment late) | 'default' (stops paying
    halfway) | 'random' (probabilities depend on the borrower's risk class).
    """
    loan = conn.execute("SELECT * FROM loans WHERE loan_id = ?", (loan_id,)).fetchone()
    if loan is None:
        raise ValidationError(f"Loan #{loan_id} does not exist.")
    if scenario not in ("all_paid", "some_late", "default", "random"):
        raise ValidationError("Unknown scenario.")
    request_id = conn.execute("SELECT request_id FROM matches WHERE match_id = ?", (loan["match_id"],)).fetchone()[0]
    probs = config.REPAYMENT_PROBABILITIES[_category_for_request(conn, request_id)]
    rng = random.Random(seed)
    for rep in conn.execute("SELECT repayment_id, installment_no FROM repayments WHERE loan_id = ? AND"
                            " status = 'pending' ORDER BY installment_no", (loan_id,)).fetchall():
        n = rep["installment_no"]
        if scenario == "all_paid":
            status = "paid"
        elif scenario == "some_late":
            status = "late" if n % 4 == 0 else "paid"
        elif scenario == "default":
            status = "defaulted" if n > loan["duration_months"] // 2 else "paid"
        else:
            status = rng.choices(["paid", "late", "defaulted"], weights=probs)[0]
        mark_installment(conn, rep["repayment_id"], status)
        if status == "defaulted":
            break
    return get_loan(conn, loan_id)


def simulate_all(conn, scenario="random", seed=None):
    rng = random.Random(seed)
    ids = [r[0] for r in conn.execute("SELECT loan_id FROM loans WHERE status = 'active' ORDER BY loan_id")]
    for loan_id in ids:
        simulate_loan(conn, loan_id, scenario, rng.randint(0, 10**9))
    return ids


# ---------------------------------------------------------------------------
# queries for screens
# ---------------------------------------------------------------------------
_LOAN_SQL = """
SELECT lo.loan_id, lo.match_id, lo.lender_id, ul.full_name AS lender, lo.borrower_id, ub.full_name AS borrower,
       lo.principal, lo.interest_rate, lo.duration_months, lo.outstanding_balance, lo.status, lo.start_date,
       (SELECT COUNT(*) FROM repayments r WHERE r.loan_id = lo.loan_id AND r.status IN ('paid','late')) AS settled
FROM loans lo
JOIN lenders l ON l.lender_id = lo.lender_id JOIN users ul ON ul.user_id = l.user_id
JOIN borrowers b ON b.borrower_id = lo.borrower_id JOIN users ub ON ub.user_id = b.user_id
"""


def list_loans(conn):
    return [dict(r) for r in conn.execute(_LOAN_SQL + " ORDER BY lo.loan_id")]


def get_loan(conn, loan_id):
    row = conn.execute(_LOAN_SQL + " WHERE lo.loan_id = ?", (loan_id,)).fetchone()
    return dict(row) if row else None


def get_repayments(conn, loan_id):
    return [dict(r) for r in conn.execute("SELECT * FROM repayments WHERE loan_id = ? ORDER BY installment_no",
                                          (loan_id,))]
def list_pending_funding(conn):
    """Fully matched requests waiting for acceptance."""
```

```
    sql = ("SELECT r.request_id, u.full_name AS borrower, r.amount, r.term_months,"
           " COUNT(m.match_id) AS lenders, ROUND(SUM(m.allocated_amount), 2) AS allocated"
           " FROM loan_requests r JOIN borrowers b ON b.borrower_id = r.borrower_id"
           " JOIN users u ON u.user_id = b.user_id"
           " JOIN matches m ON m.request_id = r.request_id AND m.status = 'recommended' AND m.allocated_amount > 0"
           " WHERE r.status = 'matched' GROUP BY r.request_id ORDER BY r.request_id")
    return [dict(r) for r in conn.execute(sql)]
```

[NEW] routes/loans.py

```
"""Week 5 routes - accept matches, loan list, repayment schedule, repayment simulation."""
from flask import Blueprint, flash, redirect, render_template, request, url_for

import loan_service as ls
from models import ValidationError
from utils import db_conn

bp = Blueprint("loans", __name__, url_prefix="/loans")


def _flash_errors(exc):
    for message in exc.errors:
        flash(message, "danger")


@bp.route("/pending")
def pending():
    with db_conn() as conn:
        rows = ls.list_pending_funding(conn)
    for row in rows:
        row["_actions"] = [{"label": "Accept & create loan(s)", "method": "post",
                            "url": url_for("loans.accept", request_id=row["request_id"])}]
    columns = [("request_id", "Request"), ("borrower", "Borrower"), ("amount", "Amount"),
               ("term_months", "Term (m)"), ("lenders", "Lenders"), ("allocated", "Allocated")]
    return render_template("table.html", title="Matched requests waiting for acceptance", columns=columns,
                           rows=rows, intro="Run matching first, then accept matches to create simulated loans.",
                           top_actions=[{"label": "Accept all", "method": "post", "url": url_for("loans.accept_all")}])


@bp.route("/accept/<int:request_id>", methods=["POST"])
def accept(request_id):
    with db_conn() as conn:
        try:
            ids = ls.accept_request(conn, request_id)
            flash(f"Request #{request_id} accepted: loan(s) {', '.join('#' + str(i) for i in ids)} created.", "success")
        except ValidationError as exc:
            _flash_errors(exc)
    return redirect(url_for("loans.loans"))


@bp.route("/accept-all", methods=["POST"])
def accept_all():
    with db_conn() as conn:
        ids, errors = ls.accept_all(conn)
    flash(f"{len(ids)} loan(s) created.", "success")
    for message in errors:
        flash(message, "danger")
    return redirect(url_for("loans.loans"))


@bp.route("")
def loans():
    with db_conn() as conn:
        rows = ls.list_loans(conn)
    for row in rows:
        row["_actions"] = [{"label": "Schedule", "method": "get", "style": "outline-primary",
                            "url": url_for("loans.detail", loan_id=row["loan_id"])}]
    columns = [("loan_id", "Loan"), ("lender", "Lender"), ("borrower", "Borrower"), ("principal", "Principal"),
               ("interest_rate", "Rate %"), ("duration_months", "Months"), ("settled", "Paid instalments"),
               ("outstanding_balance", "Outstanding"), ("status", "Status"), ("start_date", "Start")]
    return render_template("table.html", title="Loan lifecycle tracker", columns=columns, rows=rows,
                           top_actions=[{"label": "Simulate all repayments", "method": "post",
                                         "url": url_for("loans.simulate_everything")}])


@bp.route("/<int:loan_id>")
def detail(loan_id):
    with db_conn() as conn:
        loan = ls.get_loan(conn, loan_id)
        if loan is None:
            flash("Loan not found.", "danger")
            return redirect(url_for("loans.loans"))
        rows = ls.get_repayments(conn, loan_id)
    first_pending = next((r for r in rows if r["status"] == "pending"), None)
    if first_pending:
        base = url_for("loans.mark", repayment_id=first_pending["repayment_id"])
        first_pending["_actions"] = [
            {"label": "Paid", "method": "post", "style": "success", "url": f"{base}?status=paid"},
            {"label": "Late", "method": "post", "style": "warning", "url": f"{base}?status=late"},
            {"label": "Default", "method": "post", "style": "danger", "url": f"{base}?status=defaulted"}]
    columns = [("installment_no", "#"), ("due_date", "Due date"), ("amount_due", "Amount due"),
               ("amount_paid", "Paid"), ("status", "Status"), ("paid_date", "Paid on")]
    title = (f"Loan #{loan_id}: {loan['lender']} -> {loan['borrower']} | principal {loan['principal']:,.2f} | "
             f"outstanding {loan['outstanding_balance']:,.2f} | {loan['status']}")
    actions = [{"label": label, "method": "post", "style": "secondary",
                "url": url_for("loans.simulate", loan_id=loan_id, scenario=key)}
               for label, key in (("Simulate: all paid", "all_paid"), ("Simulate: some late", "some_late"),
                                  ("Simulate: default", "default"), ("Simulate: random", "random"))]
    return render_template("table.html", title=title, columns=columns, rows=rows, top_actions=actions)
@bp.route("/installments/<int:repayment_id>/mark", methods=["POST"])
```

```
def mark(repayment_id):
    with db_conn() as conn:
        loan_id = conn.execute("SELECT loan_id FROM repayments WHERE repayment_id = ?", (repayment_id,)).fetchone()
        try:
            ls.mark_installment(conn, repayment_id, request.args.get("status", ""), actor="admin")
        except ValidationError as exc:
            _flash_errors(exc)
    return redirect(url_for("loans.detail", loan_id=loan_id[0]) if loan_id else url_for("loans.loans"))


@bp.route("/<int:loan_id>/simulate", methods=["POST"])
def simulate(loan_id):
    with db_conn() as conn:
        try:
            ls.simulate_loan(conn, loan_id, request.args.get("scenario", "random"))
            flash("Repayment simulation finished.", "success")
        except ValidationError as exc:
            _flash_errors(exc)
    return redirect(url_for("loans.detail", loan_id=loan_id))


@bp.route("/simulate-all", methods=["POST"])
def simulate_everything():
    with db_conn() as conn:
        ids = ls.simulate_all(conn)
    flash(f"Simulated repayments for {len(ids)} active loan(s).", "success")
    return redirect(url_for("loans.loans"))
```

[NEW] demo_week5.py

```
"""Week 5 demo: accept matches, instalment schedules, paid / late / defaulted cases."""
from datetime import date

import demo_data
import loan_service as ls
from matching_engine import MatchingEngine

conn = demo_data.fresh_demo_db()
print("Matching:", MatchingEngine(conn).run())
loan_ids, errors = ls.accept_all(conn, start_date=date(2026, 1, 15))
print(f"\nAccepted matches -> {len(loan_ids)} loans created. Errors: {errors}")

print("\n=== Loans ===")
for l in ls.list_loans(conn):
    print(f"Loan #{l['loan_id']} {l['lender']:<19} -> {l['borrower']:<12} principal={l['principal']:>9,.0f} "
          f"rate={l['interest_rate']:g}% months={l['duration_months']:>2} outstanding={l['outstanding_balance']:>9,.0f}")

print("\n=== First 4 instalments of loan #1 ===")
for r in ls.get_repayments(conn, 1)[:4]:
    print(f"  #{r['installment_no']} due {r['due_date']} amount {r['amount_due']:>9,.2f} [{r['status']}]")

print("\n=== Loan 1: all instalments paid ===")
print(ls.simulate_loan(conn, 1, "all_paid"))
print("\n=== Loan 2: some instalments late ===")
l2 = ls.simulate_loan(conn, 2, "some_late")
print({k: l2[k] for k in ("loan_id", "status", "outstanding_balance", "settled")})
print([(r["installment_no"], r["status"], r["amount_paid"]) for r in ls.get_repayments(conn, 2)[:4]])
print("\n=== Loan 3: borrower defaults halfway ===")
l3 = ls.simulate_loan(conn, 3, "default")
print({k: l3[k] for k in ("loan_id", "status", "outstanding_balance", "settled")})
print(sorted({r["status"] for r in ls.get_repayments(conn, 3)}))

print("\n=== Lender funds after loans were created ===")
for r in conn.execute("SELECT l.lender_id, u.full_name, l.available_amount FROM lenders l JOIN users u ON u.user_id=l.user_id"):
    print(f"  {r[1]:<19} available now {r[2]:>10,.0f}")
print("\n=== Audit log (last 6 entries) ===")
for r in conn.execute("SELECT entity_type, entity_id, action, old_value, new_value FROM audit_logs ORDER BY log_id DESC LIMIT 6"):
    print(" ", tuple(r))
print("Total audit entries:", conn.execute("SELECT COUNT(*) FROM audit_logs").fetchone()[0])
conn.close()
```

[NEW] *tests/test_week5.py*

```
"""Week 5 tests: loans, schedules, repayment statuses, balances and audit log."""
import os
import sys
import unittest
from datetime import date

sys.path.insert(0, os.path.dirname(os.path.abspath(__file__)))
from helpers import add_borrower, add_lender, add_request, new_conn
import config
import loan_service as ls
from matching_engine import MatchingEngine
from models import ValidationError

START = date(2026, 1, 31)


def funded_loan(conn, amount=60000, term=12):
    lender = add_lender(conn, available=200000)
    rid = add_request(conn, add_borrower(conn), amount=amount, term=term)
    MatchingEngine(conn).run()
    loan_ids = ls.accept_request(conn, rid, start_date=START)
    return lender, rid, loan_ids[0]


class ScheduleTests(unittest.TestCase):
    def test_schedule_sums_and_balance_reaches_zero(self):
        for principal, rate, months in [(100000, 12, 12), (50000, 9, 36), (12345.67, 20, 7), (5000, 14, 3)]:
            s = ls.build_schedule(principal, rate, months, START)
            self.assertEqual(len(s), months)
            self.assertAlmostEqual(sum(i["principal"] for i in s), principal, places=2)
            self.assertEqual(s[-1]["balance_after"], 0.0)
            self.assertAlmostEqual(sum(i["amount_due"] for i in s), principal + sum(i["interest"] for i in s), places=2)

    def test_month_end_dates(self):
        s = ls.build_schedule(1000, 10, 3, START)
        self.assertEqual([i["due_date"] for i in s], ["2026-02-28", "2026-03-31", "2026-04-30"])


class AcceptTests(unittest.TestCase):
    def test_accept_creates_loan_schedule_and_updates_funds(self):
        conn, _ = new_conn()
        lender, rid, loan_id = funded_loan(conn)
        loan = ls.get_loan(conn, loan_id)
        self.assertEqual((loan["principal"], loan["duration_months"], loan["status"]), (60000, 12, "active"))
        self.assertEqual(loan["outstanding_balance"], 60000)
        self.assertIn(loan["interest_rate"], config.RATE_BY_RISK.values())
        self.assertEqual(len(ls.get_repayments(conn, loan_id)), 12)
        self.assertEqual(conn.execute("SELECT available_amount FROM lenders").fetchone()[0], 140000)
        self.assertEqual(conn.execute("SELECT status FROM loan_requests").fetchone()[0], "funded")
        self.assertEqual(conn.execute("SELECT status FROM matches WHERE status='accepted'").fetchone()[0], "accepted")

    def test_cannot_accept_twice_or_unmatched(self):
        conn, _ = new_conn()
        lender, rid, loan_id = funded_loan(conn)
        with self.assertRaises(ValidationError):
            ls.accept_request(conn, rid)
        with self.assertRaises(ValidationError):
            ls.accept_request(conn, 999)

    def test_cannot_allocate_beyond_available_funds(self):
        conn, _ = new_conn()
        lender = add_lender(conn, available=100000)
        rid = add_request(conn, add_borrower(conn), amount=60000)
        MatchingEngine(conn).run()
        conn.execute("UPDATE lenders SET available_amount = 10000")       # lender spent money elsewhere
        conn.commit()
        with self.assertRaises(ValidationError):
            ls.accept_request(conn, rid)
        self.assertEqual(conn.execute("SELECT COUNT(*) FROM loans").fetchone()[0], 0)      # nothing half-created
        self.assertEqual(conn.execute("SELECT status FROM loan_requests").fetchone()[0], "matched")

    def test_cofunded_request_creates_one_loan_per_lender(self):
        conn, _ = new_conn()
        add_lender(conn, available=60000)
        add_lender(conn, available=60000)
        rid = add_request(conn, add_borrower(conn), amount=100000)
        MatchingEngine(conn, allow_partial=True).run()
        ids = ls.accept_request(conn, rid, start_date=START)
        self.assertEqual(len(ids), 2)
        self.assertEqual(sum(ls.get_loan(conn, i)["principal"] for i in ids), 100000)


class RepaymentTests(unittest.TestCase):
    def setUp(self):
        self.conn, _ = new_conn()
        self.lender, self.rid, self.loan_id = funded_loan(self.conn)
        self.reps = ls.get_repayments(self.conn, self.loan_id)

    def test_paid_instalment_reduces_balance(self):
        ls.mark_installment(self.conn, self.reps[0]["repayment_id"], "paid")
        loan = ls.get_loan(self.conn, self.loan_id)
        expected = ls.build_schedule(60000, loan["interest_rate"], 12, START)[0]["balance_after"]
        self.assertEqual(loan["outstanding_balance"], expected)
        self.assertEqual(loan["status"], "active")
        rep = ls.get_repayments(self.conn, self.loan_id)[0]
        self.assertEqual((rep["status"], rep["amount_paid"]), ("paid", rep["amount_due"]))

    def test_late_instalment_adds_fee(self):
```

```
        ls.mark_installment(self.conn, self.reps[0]["repayment_id"], "late")
        rep = ls.get_repayments(self.conn, self.loan_id)[0]
        fee = round(rep["amount_due"] * config.LATE_FEE_RATE, 2)
        self.assertEqual(rep["amount_paid"], round(rep["amount_due"] + fee, 2))
        self.assertGreater(rep["paid_date"], rep["due_date"])

    def test_default_stops_loan(self):
        ls.mark_installment(self.conn, self.reps[0]["repayment_id"], "paid")
        ls.mark_installment(self.conn, self.reps[1]["repayment_id"], "defaulted")
        statuses = [r["status"] for r in ls.get_repayments(self.conn, self.loan_id)]
        self.assertEqual(statuses, ["paid"] + ["defaulted"] * 11)
        loan = ls.get_loan(self.conn, self.loan_id)
        self.assertEqual(loan["status"], "defaulted")
        self.assertGreater(loan["outstanding_balance"], 0)

    def test_all_paid_completes_loan(self):
        loan = ls.simulate_loan(self.conn, self.loan_id, "all_paid")
        self.assertEqual((loan["status"], loan["outstanding_balance"], loan["settled"]), ("completed", 0.0, 12))

    def test_rules_are_enforced(self):
        with self.assertRaises(ValidationError):                  # must pay in order
            ls.mark_installment(self.conn, self.reps[1]["repayment_id"], "paid")
        ls.mark_installment(self.conn, self.reps[0]["repayment_id"], "paid")
        with self.assertRaises(ValidationError):                  # cannot change twice
            ls.mark_installment(self.conn, self.reps[0]["repayment_id"], "late")
        with self.assertRaises(ValidationError):
            ls.mark_installment(self.conn, self.reps[1]["repayment_id"], "unknown")

    def test_every_status_change_is_audited(self):
        ls.simulate_loan(self.conn, self.loan_id, "some_late")
        changes = self.conn.execute("SELECT COUNT(*) FROM audit_logs WHERE entity_type='repayment'").fetchone()[0]
        self.assertEqual(changes, 12)
        self.assertEqual(self.conn.execute("SELECT COUNT(*) FROM audit_logs WHERE action='LOAN_CREATED'").fetchone()[0], 1)
        self.assertEqual(self.conn.execute("SELECT new_value FROM audit_logs WHERE entity_type='loan' AND "
                                           "action='STATUS_CHANGE'").fetchone()[0].split(" |")[0], "completed")

    def test_random_simulation_is_reproducible_and_consistent(self):
        a = ls.simulate_loan(self.conn, self.loan_id, "random", seed=7)
        conn2, _ = new_conn()
        _, _, loan2 = funded_loan(conn2)
        b = ls.simulate_loan(conn2, loan2, "random", seed=7)
        self.assertEqual((a["status"], a["settled"]), (b["status"], b["settled"]))
        paid_principal = sum(i["principal"] for i in ls.build_schedule(60000, a["interest_rate"], 12, START)[:a["settled"]])
        self.assertAlmostEqual(a["outstanding_balance"], 60000 - paid_principal if a["status"] != "completed" else 0, places=1)


class WebTests(unittest.TestCase):
    def test_screens(self):
        from app import create_app
        conn, path = new_conn()
        _, rid, loan_id = funded_loan(conn)
        conn.close()
        client = create_app(path).test_client()
        for url in ("/loans", "/loans/pending", f"/loans/{loan_id}", "/scoring/pairs", "/scoring/risk",
                    "/matching/recommendations"):
            self.assertEqual(client.get(url).status_code, 200, url)
        r = client.post(f"/loans/{loan_id}/simulate?scenario=all_paid", follow_redirects=True)
        self.assertIn(b"completed", r.data)


if __name__ == "__main__":
    unittest.main(verbosity=2)
```

Screenshots to attach to your submission

- /loans/pending before and /loans after pressing 'Accept all'.
- A loan page (/loans/1) showing the instalment schedule and the Paid / Late / Default buttons.
- The same loan after 'Simulate: some late' (late rows) and a defaulted loan.
- Terminal output of demo_week5.py and the 14 passing tests.